

IndiaOS MVP — Backend **API Specification** (FINAL CORRECTED)

****Version:**** 1.2 (Fully Architecture-Aligned)

****Status:**** Ready for Implementation

****Target:**** Next.js API Routes + Supabase

****Date:**** June 2026

Overview

IndiaOS backend provides REST API endpoints for:

- Citizen authentication (Supabase Phone Auth)
- Report submission & media upload (observations)
- Issue supporter tracking (social signal, separate from reports)
- Issue clustering (AI service, internal only)
- Governance tracking & official responses
- Public issue discovery, search & joining
- Admin platform operations (non-truth-gatekeeping)

****Architecture:****

Client (Next.js frontend) ↓ HTTPS Backend (Next.js API routes) |→ Supabase (auth, RLS, database, phone OTP) |→ Claude API (AI clustering, evidence analysis, report quality) |→ Supabase Storage (media files)

****Auth Strategy:****

- Citizens: Supabase Phone Auth (native, SMS OTP)
- Officials: Supabase Auth (email, with governance_entity_id in JWT)

- Admins: Supabase Auth (email, role=admin in JWT)
- Backend Service: Supabase service_role key (bypasses RLS for internal jobs)

****Core Philosophy (Non-Negotiable):****

- No human truth gatekeepers. AI discovers issues; evidence validates.
- Citizens never create issues directly. AI clustering does.
- Reports are observations (private inputs). Issues are clusters (public outputs).
- Supporters are affected citizens (separate from reports/observations).
- Official status ≠ Reality status (both tracked separately).

Table of Contents

1. [Base Configuration](#base-configuration)
2. [Authentication](#authentication)
3. [Citizens & Profile](#citizens--profile)
4. [Reports & Submissions](#reports--submissions)
5. [Issues & Discovery](#issues--discovery)
6. [Issue Support](#issue-support)
7. [Governance & Accountability](#governance--accountability)
8. [Evidence & Verification](#evidence--verification)
9. [Public Endpoints](#public-endpoints)
10. [Admin Endpoints](#admin-endpoints)
11. [Internal Endpoints](#internal-endpoints)
12. [Webhooks](#webhooks)
13. [Error Handling](#error-handling)
14. [Rate Limiting](#rate-limiting)

Base Configuration

****Base URL:****

https://indiaos.vercel.app/api

****Environment Variables (Backend):****

```env

NEXT\_PUBLIC\_SUPABASE\_URL=https://xxx.supabase.co

NEXT\_PUBLIC\_SUPABASE\_ANON\_KEY=eyJxxx

SUPABASE\_SERVICE\_ROLE\_KEY=eyJxxx

ANTHROPIC\_API\_KEY=sk-ant-xxx

TWILIO\_ACCOUNT\_SID=ACxxx

TWILIO\_AUTH\_TOKEN=xxx

TWILIO\_WHATSAPP\_FROM=+14155552671

### **Headers (All Authenticated Requests):**

Content-Type: application/json

Authorization: Bearer {SESSION\_TOKEN}

---

## **Authentication**

### **POST /auth/signup**

Register new citizen via Supabase Phone Auth.

#### **Request:**

```
{
 "phone_number": "+919876543210"
}
```

**Response:**

```
{
 "success": true,
 "message": "OTP sent to WhatsApp",
 "session_id": "uuid"
}
```

**Status Codes:**

- 200 — OTP sent
- 400 — Invalid phone number format
- 429 — Rate limited (max 3 OTPs per hour)

**Backend:**

```
// Use Supabase native phone auth (no custom OTP system)
const { data, error } = await supabase.auth.signInWithOtp({
 phone: phone_number,
 options: {
 shouldCreateUser: true,
 channel: 'whatsapp'
 }
});
```

---

**POST /auth/verify**

Verify OTP and create session.

**Request:**

```
{
 "phone_number": "+919876543210",
 "token": "123456"
}
```

**Response:**

```
{
 "success": true,
 "access_token": "eyJxxx",
 "refresh_token": "eyJxxx",
 "user": {
 "id": "uuid",
 "phone": "+919876543210"
 }
}
```

### **Status Codes:**

- 200 — Verified
- 400 — Invalid OTP
- 401 — Session expired

### **Backend:**

```
// Supabase handles verification
const { data, error } = await supabase.auth.verifyOtp({
 phone: phone_number,
 token: otp_token,
 type: 'sms'
});
```

```
// Auto-create citizens row on first login
if (data.user && !existingCitizen) {
 await supabase.from('citizens').insert({
 id: data.user.id,
 phone_hash: sha256(phone_number + SALT),
 verification_level: 'phone_verified',
 phone_verified_at: new Date(),
 });
}
```

```
 status: 'active'
 });
}
```

---

## **POST /auth/logout**

Invalidate session.

### **Request:**

```
{}
```

### **Response:**

```
{
 "success": true
}
```

---

## **Citizens & Profile**

### **GET /citizen/profile**

Fetch authenticated citizen's profile.

### **Request:**

GET /citizen/profile

Authorization: Bearer {ACCESS\_TOKEN}

### **Response:**

```
{
 "id": "uuid",
 "phone_hash": "sha256...",
 "verification_level": "phone_verified",
 "status": "active",
 "report_count": 42,
 "issues_supported_count": 15,
 "created_at": "2026-06-15T10:30:00Z",
}
```

```
"last_activity_at": "2026-06-28T14:22:00Z"
}
```

**RLS:** Citizen sees own row only.

---

## **GET /citizen/reports**

List authenticated citizen's reports (observations).

### **Request:**

GET /citizen/reports?limit=10&offset=0&status=submitted

### **Query Parameters:**

- limit — Max 50 (default: 10)
- offset — Pagination (default: 0)
- status — Filter: submitted|verified|disputed|archived

### **Response:**

```
{
 "reports": [
 {
 "id": "uuid",
 "title": "Pothole on Main Street",
 "description": "Large crater...",
 "category_id": 1,
 "location_id": 45,
 "status": "submitted",
 "created_at": "2026-06-25T08:00:00Z",
 "issue_id": null,
 "issue_title": null,
 "media_count": 2
 }
],
}
```

```
"total": 42,
"limit": 10,
"offset": 0
}
```

---

## **Reports & Submissions**

### **POST /reports/submit**

Submit new observation (report) with optional media.

#### **Request (multipart/form-data):**

POST /reports/submit

Authorization: Bearer {ACCESS\_TOKEN}

Content-Type: multipart/form-data

Form Data:

title: "Broken streetlight"

description: "Not working for 3 weeks"

category\_id: 2

location\_id: 45

latitude: 28.6139

longitude: 77.2090

media: [File, File, ...] (optional, max 3 files, 50MB total)

#### **Response:**

```
{
 "success": true,
 "report_id": "uuid",
 "status": "submitted",
 "media_uploaded": 2,
 "ai_analysis_queued": true,
```



```
"created_at": "2026-06-28T14:22:00Z"
}
```

#### **Status Codes:**

- 201 — Report created
- 400 — Invalid input
- 413 — Media too large
- 401 — Unauthorized

#### **Backend Flow:**

```
// 1. Validate input (coordinates, category, location exist)
// 2. INSERT reports table (citizen_id from JWT)
// 3. Upload media to Supabase Storage:
reports_media/{report_uuid}/{media_type}/{filename}
// 4. INSERT reports_media records
// 5. Trigger: update_citizen_activity_on_report() fires
// 6. Queue async job: AI report analysis
// - Spam probability (0-1)
// - Duplicate probability vs existing (0-1)
// - Quality score (0-1)
// - Saves to reports table (NOT blocking citizen)
// 7. SEPARATE: Queue clustering job (async, background)
// - Uses configurable clustering thresholds (not hardcoded >85%)
```

---

#### **GET /reports/{report\_id}**

Fetch report/observation details (citizen sees own, admins see all).

#### **Request:**

GET /reports/{report\_id}

Authorization: Bearer {ACCESS\_TOKEN}

#### **Response:**

```
{
 "id": "uuid",
 "citizen_id": "uuid",
 "title": "Pothole",
 "description": "...",
 "category_id": 1,
 "location_id": 45,
 "status": "submitted",
 "created_at": "2026-06-25T08:00:00Z",
 "issue_id": "uuid",
 "issue_title": "Potholes on Main Street",
 "media": [
 {
 "id": "uuid",
 "file_path": "reports_media/{uuid}/photo/img1.jpg",
 "media_type": "photo",
 "original_filename": "IMG_001.jpg"
 }
]
}
```

**RLS:** Citizen sees own; Admin sees all.

---

## Issues & Discovery

### GET /issues

List public issues (paginated, ranked).

#### Request:

GET /issues?location\_id=45&category\_id=2&limit=20&offset=0&sort=confidence

#### Query Parameters:

- location\_id — Filter by location (optional)
- category\_id — Filter by category (optional)
- limit — Max 100 (default: 20)
- offset — Pagination (default: 0)
- sort — confidence|momentum|recent (default: confidence)
- status — emerging|verified|trending (optional)

**Response:**

```
{
 "issues": [
 {
 "id": "uuid",
 "title": "Potholes on Main Street",
 "description": "Multiple potholes reported...",
 "category_id": 1,
 "category_name": "Roads",
 "location_id": 45,
 "location_name": "Mumbai",
 "ward_name": "Ward 42",
 "district_name": "Mumbai Metropolitan",
 "location_label": "Near Central Station, Ward 42",
 "official_status": "unacknowledged",
 "reality_status": "verified",
 "confidence_score": 0.87,
 "confidence_percent": "87%",
 "momentum_score": 0.72,
 "momentum_percent": "72%",
 "report_count": 23,
 "supporter_count": 156,
 }
]
}
```

```
"evidence_count": 4,
"created_at": "2026-06-20T10:00:00Z",
"updated_at": "2026-06-28T14:22:00Z",
"user_supported": false
}
],
"total": 156,
"limit": 20,
"offset": 0
}
```

**CRITICAL:** Public response includes:

- report\_count — Actual observations submitted (data quality)
- supporter\_count — Citizens affected (social signal)
- NO exact coordinates — Returns location hierarchy only

---

## GET /issues/search

Search issues by title/description.

### Request:

GET /issues/search?q=pothole&limit=20

### Query Parameters:

- q — Search query (min 2 chars)
- limit — Max results (default: 20, max: 100)
- location\_id — Optional location filter

### Response:

```
{
 "results": [
 {
 "id": "uuid",
```

```
"title": "Potholes on Main Street",
"category_name": "Roads",
"location_name": "Mumbai, Ward 42",
"report_count": 23,
"supporter_count": 156,
"confidence_percent": "87%",
"created_at": "2026-06-20T10:00:00Z"
}
],
"total": 4,
"query": "pothole"
}
```

---

### **GET /issues/{issue\_id}**

Fetch issue details + observations + evidence + claims.

#### **Request:**

GET /issues/{issue\_id}

Authorization: Bearer {ACCESS\_TOKEN}

#### **Response:**

```
{
 "id": "uuid",
 "title": "Potholes on Main Street",
 "description": "...",
 "category_id": 1,
 "location_id": 45,
 "location_name": "Mumbai, Ward 42",
 "official_status": "unacknowledged",
 "reality_status": "verified",
```

"confidence\_score": 0.87,  
"momentum\_score": 0.72,  
"report\_count": 23,  
"supporter\_count": 156,  
"evidence\_count": 4,  
"user\_supported": true,  
"created\_at": "2026-06-20T10:00:00Z",

"governance\_entities": [  
 {  
 "id": "uuid",  
 "name": "Municipal Corporation of Mumbai",  
 "entity\_type": "municipality",  
 "governance\_level": "city",  
 "responsibility\_type": "primary",  
 "assigned\_at": "2026-06-21T08:00:00Z",  
 "acknowledgment\_date": "2026-06-22",  
 "acknowledgment\_text": "Issue acknowledged."  
 }  
,

"issue\_updates": [  
 {  
 "id": "uuid",  
 "update\_type": "work\_started",  
 "title": "Repair crew mobilized",  
 "progress\_percent": 25,  
 "created\_at": "2026-06-25T08:00:00Z"

```
}
],

"resolution_claims": [
 {
 "claimed_status": "in_progress",
 "claim_date": "2026-06-25",
 "is_valid": true
 }
],

"evidence": [
 {
 "evidence_title": "Mumbai potholes widen",
 "evidence_type": "news_article",
 "ers_score": 0.82,
 "source_name": "The Hindu"
 }
],

"lifecycle_events": [
 {
 "event_type": "created",
 "triggered_by": "ai_clustering",
 "reason": "AI detected observations with high semantic similarity"
 }
]
}
```

---

## Issue Support

### POST /issues/{issue\_id}/support

Mark issue as affecting citizen (support, not observation).

#### Request:

```
{
 "confirmation": "I am affected by this issue"
}
```

#### Response:

```
{
 "success": true,
 "supported_at": "2026-06-28T14:22:00Z",
 "total_supporters": 157
}
```

#### Status Codes:

- 201 — Support added
- 400 — Already supporting this issue
- 404 — Issue not found
- 401 — Unauthorized

#### Backend Logic:

```
// 1. Check if citizen already supports issue
```

```
const existing = await supabase
```

```
.from('issue_supporters')
```

```
.select('id')
```

```
.eq('citizen_id', citizen_id)
```

```
.eq('issue_id', issue_id)
```

```
.single();
```



```
if (existing) {
 return res.status(400).json({ error: 'Already supporting this issue' });
}
```

```
// 2. INSERT issue_supporters (not reports)
```

```
const support = await supabase
```

```
.from('issue_supporters')
```

```
.insert({
```

```
 issue_id: issue_id,
```

```
 citizen_id: citizen_id,
```

```
 created_at: new Date()
```

```
})
```

```
.single();
```

```
// 3. UPDATE issues.supporter_count (denormalized for ranking)
```

```
await supabase.rpc('increment_issue_supporter_count', {
```

```
 issue_id: issue_id
```

```
});
```

```
// NOTE: Does NOT create a report. Support != observation.
```

```
// If citizen wants to submit observation, use POST /reports/submit
```

```
return res.status(201).json({
```

```
 success: true,
```

```
 supported_at: support.created_at,
```

```
 total_supporters: (fetch fresh count)
```

```
});
```

**Philosophy:**

- Support = "I'm affected, social signal"
  - Report = "I observed this, here's evidence"
  - Separate tables = data integrity
  - report\_count = observations (quality)
  - supporter\_count = affected citizens (reach)
- 

### **DELETE /issues/{issue\_id}/support**

Remove support from issue.

#### **Request:**

DELETE /issues/{issue\_id}/support

Authorization: Bearer {ACCESS\_TOKEN}

#### **Response:**

```
{
 "success": true,
 "removed_at": "2026-06-28T14:22:00Z"
}
```

---

## **Governance & Accountability**

### **GET /governance/entities**

List government entities (public).

#### **Request:**

GET /governance/entities?governance\_level=city&location\_id=45

#### **Response:**

```
{
 "entities": [
 {
 "id": "uuid",
 "name": "Municipal Corporation of Mumbai",

```

```
"entity_type": "municipality",
"governance_level": "city",
"location_id": 45,
"contact_email": "grievances@mcgm.gov.in",
"website_url": "https://www.mcgm.gov.in"
}
]
}
```

---

### **POST /governance/issue-updates**

Post operational progress update (official only).

#### **Request:**

```
{
 "issue_id": "uuid",
 "update_type": "work_started",
 "title": "Repair crew mobilized",
 "description": "5-person crew dispatched",
 "progress_percent": 25,
 "evidence_id": null
}
```

#### **Response:**

```
{
 "success": true,
 "id": "uuid",
 "created_at": "2026-06-28T14:22:00Z"
}
```

---

### **POST /governance/resolution-claims**

Claim issue resolved (official only).

**Request:**

```
{
 "issue_id": "uuid",
 "claimed_status": "completed",
 "claim_evidence_url": "https://govsite.com/tender-award-123"
}
```

**Response:**

```
{
 "success": true,
 "id": "uuid",
 "claim_number": 1,
 "claim_date": "2026-06-28",
 "created_at": "2026-06-28T14:22:00Z"
}
```

**Backend Logic:**

```
// 1. INSERT resolution_claims (government's assertion)

// 2. Snapshot: reality_status_at_claim = current issues.reality_status (IMMUTABLE)

// 3. Set official_status = claimed_status

// 4. Background job monitors: Reality Gap Score (RGS)

// Threshold configurable via governance_terms (not hardcoded >0.60)

// If RGS exceeds configured threshold, reopen: set official_status='in_progress'

// Fire lifecycle event: triggered_by='rgs_reopen'
```

---

**Evidence & Verification**

**GET /issues/{issue\_id}/evidence**

List evidence items for issue.

**Request:**

GET /issues/{issue\_id}/evidence?limit=20

**Response:**

```
{
 "evidence": [
 {
 "id": "uuid",
 "evidence_url": "https://...",
 "evidence_title": "Mumbai potholes widen",
 "evidence_type": "news_article",
 "ers_score": 0.82,
 "published_at": "2026-06-22",
 "source_name": "The Hindu"
 }
]
}
```

**RLS:** Public (all can see evidence).

---

**Public Endpoints**

**GET /categories**

List issue categories.

**Request:**

GET /categories

**Response:**

```
{
 "categories": [
 {
 "id": 1,
 "name": "Roads",

```

```
 "description": "Potholes, broken asphalt",
 "sla_days_benchmark": 30
 }
]
}
```

---

### **GET /locations**

List locations (hierarchical).

#### **Request:**

GET /locations?parent\_location\_id=1

#### **Response:**

```
{
 "locations": [
 {
 "id": 45,
 "name": "Mumbai",
 "location_type": "city",
 "parent_location_id": 1,
 "population": 20961472
 }
]
}
```

---

### **GET /health**

Health check (public, no auth).

#### **Request:**

GET /health

#### **Response:**

```
{
 "status": "ok",
 "database": "connected",
 "timestamp": "2026-06-28T14:22:00Z"
}
```

---

## Admin Endpoints

### GET /admin/dashboard

Admin dashboard stats (admin only).

#### Request:

GET /admin/dashboard?date\_from=2026-06-01&date\_to=2026-06-28

Authorization: Bearer {ADMIN\_TOKEN}

#### Response:

```
{
 "summary": {
 "total_citizens": 2150,
 "active_citizens": 1840,
 "total_reports": 12500,
 "total_issues": 450,
 "issues_resolved": 120,
 "avg_resolution_days": 18.3,
 "total_supporters": 18500
 },
 "daily_metrics": [
 {
 "date": "2026-06-28",
 "reports_submitted": 42,
 "supporters_added": 127,
```

```
"issues_created": 3
}
],
"clustering": {
 "reports_processed_24h": 1247,
 "new_issues_created": 12
}
}
```

---

## Internal Endpoints

### POST /internal/cluster-reports

Cluster unlinked reports into issues (service\_role key only).

#### Request (Backend Internal):

```
{
 "batch_size": 50,
 "use_semantic_search": true
}
```

#### Backend Flow:

```
// 1. Fetch unlinked reports (issue_id IS NULL, archived=false)
// 2. For each report, compute semantic embedding via Claude API
// 3. Fetch clustering threshold from governance config (configurable, not hardcoded)
// 4. Cluster by similarity (>= configured threshold, e.g., 0.85)
// 5. For each cluster:
// a. CREATE issue (if new cluster)
// b. UPDATE reports.issue_id for all in cluster
// c. INSERT issue_lifecycle_events (triggered_by='ai_clustering')
// 6. Triggers auto-fire: counters update, timestamps refresh
```

#### Response:



```
{
 "success": true,
 "reports_processed": 47,
 "new_issues_created": 3,
 "reports_linked": 42,
 "processing_time_ms": 8234
}
```

---

### **POST /internal/compute-scores**

Recompute confidence, momentum, RGS scores (daily batch).

#### **Request:**

```
{
 "target_issues": "all",
 "refresh_lifecycle": true
}
```

#### **Backend:**

```
// 1. For each issue:
// a. Recompute confidence_score (CS) via Claude API
// b. Recompute momentum_score (recent supporter + report velocity)
// c. Recompute evidence_count
// 2. For completed issues:
// a. Compute Reality Gap Score (RGS) vs claims
// b. Fetch RGS reopening threshold from governance config (configurable, not
// hardcoded)
// c. If RGS >= threshold, reopen: set official_status='in_progress'
// d. Fire lifecycle event (triggered_by='rgs_reopen')
// 3. Update issues.updated_at
```

#### **Response:**

```
{
 "success": true,
 "issues_scored": 450,
 "issues_reopened": 12
}
```

---

### **POST /internal/report-analysis**

Analyze report quality (AI analysis, not human gatekeeping).

#### **Request (internal trigger):**

```
{
 "report_id": "uuid"
}
```

#### **Response:**

```
{
 "success": true,
 "analysis": {
 "spam_probability": 0.05,
 "duplicate_probability": 0.15,
 "quality_score": 0.92,
 "flagged_for_review": false,
 "reason": null
 },
 "updated_at": "2026-06-28T14:22:00Z"
}
```

#### **Backend:**

```
// Call Claude API to analyze (AI performs analysis)
const analysis = await anthropic.messages.create({
 model: 'claude-opus-4-6',
```

max\_tokens: 500,

system: `Analyze this citizen observation for:

1. Spam probability (0-1): promotional, nonsense, irrelevant
2. Duplicate probability (0-1): likely already reported
3. Quality score (0-1): clarity, actionability, detail

Return JSON: { spam\_probability, duplicate\_probability, quality\_score }` ,

messages: [

{ role: 'user', content: `Title: \${report.title}\nDescription: \${report.description}` }

]

});

// Store in database (reference only, not truth gate)

// Admins REVIEW these scores, not use them to block reports

**Philosophy:** AI flags suspicious reports; admins review and investigate (not judge truth).

---

## Webhooks

### POST /webhook/twilio/incoming-message

Twilio WhatsApp webhook (incoming messages from citizens).

#### Twilio Sends (raw HTTP POST):

POST /webhook/twilio/incoming-message

Content-Type: application/x-www-form-urlencoded

MessageSid=SMxxx

From=whatsapp:+919876543210

To=whatsapp:+14155552671

Body=Pothole on Main Street

NumMedia=1

MediaUrl0=https://api.twilio.com/...

**Backend Logic:**

```
// 1. Validate Twilio signature
// 2. Extract phone number
// 3. Find citizen (via phone_hash)
// 4. If not authenticated: send OTP link
// 5. If authenticated:
// a. INSERT bot_conversation record
// b. Extract media (if any)
// c. Queue for report creation / suggestion
```

**Response (Twilio expects 200 OK):**

200 OK

{}

---

**POST /webhook/twilio/message-status**

Twilio webhook for message delivery.

**Twilio Sends:**

POST /webhook/twilio/message-status

MessageSid=SMxxx

MessageStatus=delivered

---

**Error Handling****Standard Error Response:**

```
{
 "error": {
 "code": "INVALID_INPUT",
 "message": "latitude must be between -90 and 90",
 "details": {
 "field": "latitude",
```

```
"value": 95.5,
 "constraint": "range"
}
}
}
```

## HTTP Status Codes:

### Code Meaning

|     |                              |
|-----|------------------------------|
| 200 | Success                      |
| 201 | Created                      |
| 400 | Bad Request (validation)     |
| 401 | Unauthorized (missing auth)  |
| 403 | Forbidden (RLS policy)       |
| 404 | Not Found                    |
| 409 | Conflict (unique constraint) |
| 429 | Rate Limited                 |
| 500 | Server Error                 |

---

## Rate Limiting

### Per Client (identified by IP + user\_id):

| Endpoint                  | Limit Window |        |
|---------------------------|--------------|--------|
| POST /auth/signup         | 3            | 1 hour |
| POST /reports/submit      | 10           | 1 day  |
| POST /issues/{id}/support | 50           | 1 day  |
| GET /issues               | 1000         | 1 hour |
| GET /issues/search        | 100          | 1 hour |

| Endpoint           | Limit Window |
|--------------------|--------------|
| POST /governance/* | 100 1 day    |

---

## Security Notes

1. **HTTPS Only** — All endpoints require HTTPS.
  2. **CORS** — Allow next.js frontend origin only.
  3. **Phone Authentication** — Supabase native phone auth (no custom OTP system).
  4. **Service Role Security** — Backend key never exposed; used for internal jobs only.
  5. **Coordinate Privacy** — Public issue APIs return city/district/ward/label only; exact coordinates stored internally.
  6. **RLS Enforcement** — All queries flow through Supabase RLS policies.
  7. **AI-Driven, Not Human-Gated** — AI analyzes reports; admins manage operations, not truth.
  8. **Data Integrity** — Reports and supporters tracked separately (report\_count ≠ supporter\_count).
  9. **Configurable Thresholds** — Clustering confidence and RGS reopening thresholds configurable via governance\_terms, not hardcoded.
- 

## Database Schema Notes

### New Table: issue\_supporters

```
CREATE TABLE issue_supporters (
 id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
 issue_id UUID NOT NULL FOREIGN KEY REFERENCES issues(id) ON DELETE
 CASCADE,
 citizen_id UUID NOT NULL FOREIGN KEY REFERENCES citizens(id) ON DELETE
 CASCADE,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

 CONSTRAINT uc_issue_supporter UNIQUE (issue_id, citizen_id)
```

```
);
```

```
CREATE INDEX idx_issue_supporter_issue ON issue_supporters(issue_id);
```

```
CREATE INDEX idx_issue_supporter_citizen ON issue_supporters(citizen_id);
```

### **Modified: issues table**

```
-- Add column:
```

```
ALTER TABLE issues ADD COLUMN supporter_count INT DEFAULT 0;
```

```
-- Add constraint:
```

```
ALTER TABLE issues ADD CONSTRAINT issues_supporter_count_nonnegative CHECK
(supporter_count >= 0);
```

### **Trigger: Maintain supporter count**

```
CREATE FUNCTION update_issue_on_supporter_add()
```

```
 RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
 UPDATE issues
```

```
 SET supporter_count = supporter_count + 1
```

```
 WHERE id = NEW.issue_id;
```

```
 RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_update_issue_on_supporter_add
```

```
 AFTER INSERT ON issue_supporters
```

```
 FOR EACH ROW
```

```
 EXECUTE FUNCTION update_issue_on_supporter_add();
```

```
-- Similar for DELETE
```

---

**API Specification Complete — Fully Architecture-Aligned, Data-Integrity Preserved,  
Ready for Implementation**